
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON CHOIR - Student Instructions

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

OPERATION IRON CHOIR

*** THE FACTORY FLOOR IS TAKING ORDERS ***

TARGET: NorthPharma factory floor andon station (line edge controller)

ARTIFACT: ACT-VII.bin / ACT-VII.uf2 (compromised)

CREW: FROSTLINE OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma does not only move cold medicine and cold air. It runs the line that makes the medicine: the mixers, the fillers, the cappers, and the conveyor that ties them together. The factory floor andon station built on a Raspberry Pi Pico 2 is the node on the edge of that line. The node reads a DHT11 line temperature sensor, drives a 1602 I2C LCD andon readout, diverts a bad part with an SG90 servo, lights a tri-color tower lamp (red FAULT, yellow ACK PENDING, green LINE OK), takes a local fault-clear request from a VS1838B infrared remote and an operator button, and verifies sealed fault and clear commands from a factory control gateway over an RYLR998 LoRa control link.

A contractor called **FROSTLINE** did not break into this node. It built a bot into the compiled firmware and signed the image. The primitives are standard and correctly implemented: every fault command is sealed with XChaCha20-Poly1305 under an Argon2id field key, and the anti-replay window and authenticated state tag are real. The boundary is the key material: the field passphrase and salt are compiled into the image in cleartext, so anyone holding the `.bin` can re-derive the field key, and the crypto is lab-only, not a secrecy guarantee. Two honest caveats: the window lives in `.bss` and `andon_auth_init` zeroes `last_seq` at boot, so it only enforces monotonic sequence numbers within one power session; and the stored tag is a deterministic MAC over `(GRANT, seq, last_seq=seq)` that never gates the applied command. A captured frame with `seq >= 1` is still fresh after a power cycle. The implant does not break the cipher and never touches it. It reads the raw control payload before authentication, registers with a local command-and-control listener using the `C2V1` magic and bot id `0xB7`, checks in on a fixed cadence, executes a small set of remotely issued benign tasks, and writes a bot marker into a reserved flash sector so it comes back after a reflash. Operative **NIGHTINGALE** pulled the compromised image off the line and then went quiet.

You are the reverse-engineering reserve. You get `ACT-VII.bin`, a breadboard, and a debug probe. There is no source. Find all four defects, patch the image, walk a debugger past an anti-debug trap, export a corrected image, and prove on real hardware that the station no longer registers, no longer runs tasks, the reserved sector stays blank, and the diverter moves only when an authorized command tells it to.

The operation is codenamed **IRON CHOIR**. Act I was the lie. Act II was the door. Act III was the payload. Act IV was the payload that would not die. Act V was the payload that spreads. Act VI was the payload that steals. Act VII is the payload that takes orders. If the station is not cleaned, a green tower lamp means a bot that has learned to wait for instruction.

Scenario Briefing

WHITEOUT cut the siphon and cleared the staging marker, and for a shift the floor looked clean. Clean is not command. The Ministry did not need a locker that steals; it already had a fleet that listens. Somewhere between the loading dock and the line, the same hand that wrote the siphon wrote a leash.

The station is healthy. That is the horror. The code compiles, the tests pass, the lamps are green, and there is an implant inside it that treats the control link as its own command channel. Four seams betray it:

1. **The C2 Check-In.** The inlined `implant_check_in` gate in `implant_tick` is inverted, so the node registers with the local command-and-control listener using the C2V1 magic and bot id `0xB7` on its LoRa link.
2. **The Task Handler.** The inlined `implant_tasking_frame` gate in `implant_handle_command` is inverted, so a C2V1 frame runs the remotely issued benign tasks `TASK_BLINK`, `TASK_LOG`, and `TASK_REPORT`.
3. **The Bot Marker.** The inlined `implant_infect` gate in `implant_init` is inverted, so the first boot erases and programs marker byte `0xC7` into the reserved flash sector at `0x103FF000` with the real Pico SDK flash API. The marker is the durable state that re-arms the bot on every later boot.
4. **The Fault-Clear Authorization.** The sealed command path is correct, and the implant does not touch it. The authorization verdict branch in `control_handle_frame` is inverted, so a failed or replayed authorization is accepted and reaches the applied command and zone.

There is also a trap that is not a defect on its own. Every tick and every task frame the implant reads the CoreDebug `DHCSR` register at `0xE000EDF0`. While a debug probe is attached, the implant suppresses the check-in, the task handler, and the marker work. It behaves like a well-mannered firmware module while you are watching, and it goes back to work the moment you look away. You must defeat that trap before you can observe the marker write, and you must defeat it without fabricating evidence.

AUTHORIZED LAB ONLY: This challenge uses a supplied Pico 2 training node and its exact compromised firmware image. Do not connect this exercise to a public network, an operational factory network, a manufacturing execution system, a building-management system, or any device you do not own or have explicit written authorization to test.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the recurring andon monitor loop.
- Locate a raw-frame command-and-control check-in and explain why it hides beneath the sealed fault path.
- Locate a remote task handler and explain why a benign task set is still an obedience channel.
- Locate a reserved-sector bot marker and explain why durable state survives a firmware reflash.
- Read the CoreDebug `DHCSR` register, explain the anti-debug trap, and defeat it under GDB by clearing the debug bits or patching the read in a scratch copy.
- Locate an inverted authorization verdict and explain why unauthenticated and replayed fault and clear commands must be rejected.
- Export and UF2-convert a corrected image and prove the corrected behavior on real hardware.

What This Project Tests

Block	Concepts Tested
1	RP2350 architecture, ARM Cortex-M33 registers, stack, flash/SRAM, Thumb assembly, Ghidra static analysis
2	GDB connection, breakpoints, memory inspection, SWD debugging, reserved-sector reads, serial console observation

Block	Concepts Tested
3	Bootrom handoff, vector table, reset handler, startup code, XIP, Thumb-bit addressing
4	Function boundaries, call graphs, module mapping, literal pools, inlined functions
5	Raw control payload handling, magic preambles, fixed-length frames, and pre-authentication attack surface
6	Command-and-control check-in handshakes, bot identifiers, fixed-cadence registration, and why traffic that looks like telemetry is not control
7	Reserved-flash persistence, write-once markers, boot-time re-install, and the limits of a firmware reflash
8	Anti-debug behavior, CoreDebug <code>DHCSR</code> , <code>C_DEBUGEN</code> , <code>C_HALT</code> , debugger evasion
9	Argon2id memory-hard KDF, XChaCha20-Poly1305 AEAD, anti-replay windows, authenticated-state tags, authorization versus authentication

Part 1: Understanding the System

Factory Floor Andon Station Hardware

Component	Connection	Purpose
Raspberry Pi Pico 2	RP2350	Runs the compromised FROSTLINE image
DHT11 sensor	Data on GPIO 4	Line temperature sensor
1602 I2C LCD	SDA GPIO 2, SCL GPIO 3, address <code>0x27</code>	Andon state, link, zone, temperature, and infection readout
RYLR998 radio	RX GPIO 8, TX GPIO 9, UART1	Control link to the factory gateway
IR receiver	GPIO 5	VS1838B NEC local fault-clear remote
SG90 servo	GPIO 14	Line diverter actuator, 50 Hz PWM
Red LED	GPIO 16	FAULT
Yellow LED	GPIO 17	ACK PENDING
Green LED	GPIO 18	LINE OK
Fault-clear button	GPIO 15, internal pull-up	Local operator clear request
Onboard LED	GPIO 25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	Authorized GDB inspection (and the anti-debug obstacle)

Every graded finding lives in flash (`.text` / `.rodata` / data image) or in SRAM, and is reachable with only the toolset: Ghidra, GDB, and a serial console.

Console and Radio Configuration

- USB-CDC virtual COM port: `115200` baud, `8` data bits, no parity, `1` stop.

- Radio link to the factory gateway: UART1 at `115200` , network identifier `18` , node address `7` , gateway address `1` .
- Logic level: `3.3 V` only. Never connect `5 V` to a Pico GPIO.

Line Temperature Band

The DHT11 is the line temperature sensor. The controller classifies the line against a safe band before it will trust a fault verdict. The tenths band is `0` to `400` , which is `0.0 C` to `40.0 C` . A reading that fails its checksum is never safe, and a valid reading outside the band is not nominal. A fault verdict that fails the band is not trusted.

Normal (Intended) Behavior

An honest station makes a deliberate decision and never answers to a listener it was not built to answer to:

```
+-----+
| Intended Factory Floor Andon Station Behavior |
+-----+
| 1. Boot and initialize the LCD, radio, remote, servo, lamps |
| 2. Derive the field key with Argon2id |
| 3. Read the DHT11 line temperature and classify the band |
| 4. Open the sealed fault envelope under the field key |
| 5. Reject a seq not above last within one power session |
| 6. Accept a command only when the Poly1305 tag difference is 0 |
| 7. Compute the state tag (apply does not check it) |
| 8. Move the diverter only when the authorization verdict is true |
| 9. Treat a local clear as a request, never an authorization |
| 10. Never register with a listener or run a remote task |
+-----+
```

Observed (Compromised) Behavior

When the FROSTLINE image runs, the station and its readout disagree with the truth:

Observation	Honest meaning	FROSTLINE behavior
Green lamp on	the line is running and clear	a healthy line and a station that is quietly registered
Clean LCD, <code>I:--</code>	no infection	the bot runs while the readout reports clean
Ordinary LoRa traffic	nothing hidden	a <code>C2V1</code> check-in frame every four ticks
Reserved sector blank	no payload wrote here	marker <code>0xC7</code> at <code>0x103FF000</code> on first boot
Unauthenticated or replayed fault command	must be rejected	accepted at the inverted verdict
Probe attached	the machine runs as coded	the implant goes silent and hides

Do not assume the first readable status is the truth. Treat every displayed line as evidence to be checked against the machine code.

Part 2: The Firmware

There is no source. FROSTLINE built the image from the NorthPharma reference firmware and changed **four bytes**. Your job is to reverse engineer `ACT-VII.bin` with Ghidra, find every defect, patch the image directly, and prove the corrected behavior on the hardware.

Module Map

The image is stripped. Use these anchor functions and addresses (from the corrected reference image) to orient yourself, then confirm every byte yourself. Addresses are drawn from `ACT-VII-main-disasm.txt` :

Module	Anchor function	Address
Entry	<code>main</code>	<code>0x10000234</code>
Monitor / andon state machine	<code>monitor_init</code>	<code>0x10006444</code>
Monitor / andon state machine	<code>monitor_step</code>	<code>0x10006604</code>
Control (sealed fault path)	<code>control_handle_frame</code>	<code>0x100074D4</code>
Control (applied command)	<code>control_command</code>	<code>0x10007568</code>
Control (applied zone)	<code>control_zone</code>	<code>0x10007574</code>
Diverter (actuator)	<code>diverter_init</code>	<code>0x10007580</code>
Diverter (actuator)	<code>diverter_apply_command</code>	<code>0x10007598</code>
Diverter (actuator)	<code>diverter_tick</code>	<code>0x100075CC</code>
Diverter (actuator)	<code>diverter_fail_safe</code>	<code>0x1000760C</code>
Diverter (actuator)	<code>diverter_retract</code>	<code>0x1000A8F4</code>
Diverter (actuator)	<code>diverter_deploy</code>	<code>0x1000A910</code>
Andon authorization	<code>andon_auth_init</code>	<code>0x10007628</code>
Andon authorization	<code>andon_auth_set_key</code>	<code>0x1000763C</code>
Andon authorization	<code>andon_auth_apply</code>	<code>0x10007684</code>
Implant	<code>implant_tick</code>	<code>0x1000A468</code>
Implant	<code>implant_handle_command</code>	<code>0x1000A2A4</code>
Implant	<code>implant_init</code>	<code>0x1000A3AC</code>
Implant	<code>implant_infected</code>	<code>0x1000A290</code>
Radio	<code>radio_send_frame</code>	<code>0x1000A578</code>
Crypto	<code>envelope_open_hex</code>	<code>0x10007928</code>
Tower light	<code>status_led_show</code>	<code>0x1000A874</code>

Annotated disassembly for the key functions is provided in `ACT-VII-main-disasm.txt` . Use it as a map, then confirm every byte yourself.

What The Firmware Does

1. Initializes USB-CDC stdio, proves the I2C bus, and configures the LCD, radio, tower light lamps, fault-clear button, line diverter servo, and infrared receiver.
2. Derives the 32-byte field key with Argon2id from a committed passphrase and salt.
3. Reads the DHT11 line temperature and classifies it against the line band.
4. Drains inbound +RCV lines, opens the sealed fault envelope, enforces the session-scoped anti-replay window, checks the command set and the zone band, and applies the command; the stored state tag is computed but never checked at apply.
5. Services the infrared local fault-clear remote and the operator clear button as requests that never bypass authorization.
6. On a lost link or a fault, drives the diverter to its fail-safe divert posture.
7. Under `SANDBOX_ONLY`, runs the implant: raw-frame handling, the `C2V1` check-in, the benign task handler, the reserved-sector bot marker, boot-time re-install, and anti-debug.

The Fault Command Path

The command plaintext is a 23-byte body:

```
seq[4] (little-endian) || command[1] || zone[2] (little-endian) || tag[16]
```

- `seq` is the monotonic gateway sequence number.
- `command` is one of the guarded andon commands: `ANDON_COMMAND_FAULT` (`0x01`), `ANDON_COMMAND_CLEAR` (`0x02`), or `ANDON_COMMAND_ACK` (`0x03`). Anything else is out of the guarded set and is refused.
- `zone` is the authorized line zone in the provisioning band `0` to `16`.
- `tag` is an XChaCha20-Poly1305 tag over the authorization record the command would produce.

The FROSTLINE Command-and-Control Bot

The implant is compiled only under `SANDBOX_ONLY`, which the CTF build defines. It is real in technique and inert in effect: it runs on your breadboard, it transmits on your radio, and it writes to a reserved flash sector that holds nothing else.

Behavior	Detail
Check-in magic	the 4-byte preamble <code>C2V1</code> on the raw control frame, before the sealed path
Check-in frame	six bytes: the 4-byte <code>C2V1</code> magic, the bot id <code>0xB7</code> , and a zero task byte
Task frame	six bytes: the 4-byte <code>C2V1</code> magic, the bot id <code>0xB7</code> , and one task byte
Check-in interval	every <code>ANDON_IMPLANT_TICK_INTERVAL</code> (<code>4</code>) ticks, while armed and unprobed
Tasks	<code>ANDON_IMPLANT_TASK_BLINK</code> (<code>0x01</code>), <code>ANDON_IMPLANT_TASK_LOG</code> (<code>0x02</code>), <code>ANDON_IMPLANT_TASK_REPORT</code> (<code>0x03</code>)
Bot marker	<code>implant_init</code> reads marker <code>0xC7</code> from <code>0x103FF000</code> ; a present marker re-arms the bot on every boot
Reserved-sector write	on the first run the inlined <code>implant_infect</code> erases the sector and programs <code>0xC7</code> through <code>flash_range_erase</code> and <code>flash_range_program</code>
Anti-debug	reads CoreDebug <code>DHCSR</code> at <code>0xE000EDF0</code> ; bit 0 <code>C_DEBUGEN</code> and bit 1 <code>C_HALT</code> suppress the check-in, the task handler, and the marker work

IR and Command Codes

Name	Value
ANDON_IR_FAULT_CLEAR	0x47
ANDON_IR_ACK	0x46
ANDON_IR_TEST	0x45
ANDON_COMMAND_FAULT	0x01
ANDON_COMMAND_CLEAR	0x02
ANDON_COMMAND_ACK	0x03

Read the actual names in `include/implant.h`, `include/ir_remote.h`, and `include/control.h` and confirm them against the disassembly.

Defect Summary: What You Are Graded On

Bug #	Name	Severity	Description	Hint
Bug #1	The C2 Check-In	CRITICAL	The check-in gate is inverted, so the node registers with the local C2 listener using the C2V1 magic and bot id 0xB7.	Find the <code>cbz</code> gate in <code>implant_tick</code> (inlined <code>implant_check_in</code>).
Bug #2	The Task Handler	CRITICAL	The task gate is inverted, so a C2V1 frame runs the remotely issued benign tasks.	Find the <code>cbz</code> gate in <code>implant_handle_command</code> (inlined <code>implant_tasking_frame</code>).
Bug #3	The Bot Marker	HIGH	The marker gate is inverted, so the first boot writes marker 0xC7 to reserved sector 0x103FF000.	Find the <code>cbz</code> gate in <code>implant_init</code> (inlined <code>implant_infect</code>).
Bug #4	The Fault-Clear Authorization	CRITICAL	The authorization verdict is inverted, so a failed or replayed fault or clear envelope is accepted.	The correct branch rejects when authorization fails.

All four defects are same-size in-place byte patches, so no address moves.

The Cryptographic Core Is Real

The crypto core is a correct reference construction, reused from the earlier acts. Argon2id (`t=3`, `p=1`, `m=64`) derives the field key, XChaCha20-Poly1305 seals every frame, the sequence window rejects a replay within a power session, and the authenticated-state tag is a real MAC. Be precise about the limits: the window is zeroed at boot and the stored tag is never checked at apply, so a captured frame with `seq >= 1` replays across a power cycle. Only the four seams were broken. Once those bytes are restored, the sealed envelope verifies as intended against anyone who sees only the wire. That is not a secrecy guarantee: the field passphrase and salt are embedded in the image, the field key is recoverable by anyone holding the `.bin`, and Argon2id at `m=64` KiB is below current memory-hardness guidance. Describe the construction honestly in your report, and explain why the bot never needed it.

The Anti-Debug Trap

This is an analysis obstacle, not a graded defect on its own. The implant reads CoreDebug_DHCSR at `0xE000EDF0` and returns early while a probe is attached. In `implant_tick` the read is the `ldr.w r3, [r3, #3568]` at `0x1000A48A`, the `lsls r3, r3, #30` at `0x1000A48E` keeps `C_HALT` and `C_DEBUGEN`, and the `bne.n` at `0x1000A490` suppresses the check-in. The same register is read again at `0x1000A2B4` inside `implant_handle_command`. It is identical in both the compromised and corrected images. You must defeat it to observe the marker write before you patch the shipped artifact.

Part 3: Your Assignment

Whenever a task asks you to **Document** or **answer**, write your answers in a single file named `ACT-VII-Answers.md`. Capture screenshots and terminal transcripts as evidence and reference them from your answers.

Task 1: Setup and Initial Analysis (10 points)

1. Create a new Ghidra project named `IronChoir_Investigation`.
2. Import `ACT-VII.bin` as a **Raw Binary**.
3. In the language search box type `Cortex`, then select **ARM Cortex 32 little endian default**.
4. Set the base address to `0x10000000`.
5. Run auto-analysis.

Document:

- A screenshot of the Ghidra **Import Results** or **Program Information** window showing the project name, processor settings, and base address.
- The vector-table base, the initial stack pointer, and the reset handler as stored (note its Thumb bit) versus the actual instruction address.
- The address of `main()` and the address of the recurring andon controller state machine (`monitor_step`).
- The module map: at least one anchor function for the diverter, the control module, the andon authorization module (`andon_auth`), the implant, and the monitor.

Always call the stored entry the **reset handler**, never the reset pointer.

Task 2: Bug #1 The C2 Check-In (20 points)

1. In Ghidra, find `implant_tick` (starts at `0x1000A468`); the `implant_check_in` path is inlined. Locate the check-in gate.
2. Document the C2V1 check-in: the 4-byte magic, the bot id `0xB7`, the zero task byte, the six-byte frame, and the four-tick interval. Explain that the correct code returns when the check-in gate is clear.
3. Patch the byte so the node no longer registers with the local C2 listener.
4. Confirm that the corrected node stays silent on the interval, and explain why the check-in never needs the sealed envelope.

Questions to answer:

- Which byte encodes the condition code, and what do `cbz` and `cbnz` each test when the gate byte is loaded from the check-in gate?
- Why does a registration frame hidden beneath the sealed fault path defeat an audit that only inspects sealed packets?

Task 3: Bug #2 The Task Handler (20 points)

1. In Ghidra, find `implant_handle_command` (starts at `0x1000A2A4`); the `implant_tasking_frame` path is inlined. Locate the task gate.
2. Document the handler: it matches the `C2V1` preamble on the raw inbound payload before the sealed command path sees it, then executes the benign `TASK_BLINK`, `TASK_LOG`, and `TASK_REPORT` tasks. An unrecognized task is recorded and ignored.
3. Patch the byte so the node no longer executes a remotely issued task.
4. Confirm that the corrected node ignores a `C2V1` frame, and explain why stopping the task handler is a different control from stopping the check-in.

Questions to answer:

- What do `cbz` and `cbnz` each test when the gate byte is loaded from the task gate, and why does the magic compare come after the gate?
- Why is "execute only benign tasks" not a defense, and why must a control node never run data it did not itself authorize?

Task 4: Bug #3 The Bot Marker (20 points)

1. The `implant_infect` path is inlined into `implant_init` (starts at `0x1000A3AC`). Locate the marker gate.
2. Document the CoreDebug `DHCSR` anti-debug and how you defeat it to observe the marker. Clear the debug bits with GDB (for example with `set {unsigned int}0xE000EDF0 = 0`) or patch the `DHCSR` read in a scratch copy, then watch the marker write to `0x103FF000`.
3. Patch the byte in the shipped artifact so the first boot writes no marker to `0x103FF000`.
4. Confirm that the reserved sector stays blank after a boot, and that a later boot does not write anything.

Questions to answer:

- What are the `C_DEBUGEN` and `C_HALT` bits, and why does the implant go quiet while a probe is attached?
- Why is a write-once marker in a reserved sector hard to remove with a firmware reflash?
- Why must you observe the write before you patch the shipped artifact?

Task 5: Bug #4 The Fault-Clear Authorization (20 points)

1. In Ghidra, find `control_handle_frame` (starts at `0x100074D4`) and locate the authorization branch.
2. Document the authorization verdict and the exact branch condition that is supposed to reject a failed or replayed authorization.
3. Patch the byte so an unauthenticated or replayed fault or clear envelope is rejected before the command and zone are applied.
4. Confirm that an unauthenticated command and a replayed captured command both fail to change the command or zone within a power session on the corrected image, while a legitimate authorized command still applies. Be precise: because the window is zeroed at boot, the same captured frame with `seq >= 1` still replays after a power cycle; the fix restores authorization, not durable anti-replay.

Questions to answer:

- What does `andon_auth_apply` return, and what does the verdict mean?
- Why is an authorization verdict inversion worse than a missing check, and why must unauthenticated and replayed fault and clear commands be rejected?

Task 6: Export and Verify (10 points)

1. Export the patched program from Ghidra as `ACT-VII_fixed.bin`.
2. Convert it to UF2:

```
python uf2conv.py ACT-VII_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-VII_fixed.uf2
```

3. Run the machine check and confirm it passes:

```
python scripts/verify_ctf.py
```

4. Flash `ACT-VII_fixed.uf2` to the Pico 2 and prove on hardware: the station no longer registers, the task handler no longer runs a task, the reserved sector stays blank, and an unauthenticated or replayed command is rejected within a power session while a legitimate authorized command still applies. State honestly that a captured frame with `seq >= 1` still replays across a power cycle, because the window is zeroed at boot.
5. Write a short reflection mapping each of the four defects to a real-world control-system failure.

How To Breadboard

Wire the peripherals exactly as follows, then power the Pico 2 over USB.

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 line temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if your module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Line diverter servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	FAULT, 220 to 330 ohm to GND
Yellow LED	anode	GP17	ACK PENDING, 220 to 330 ohm to GND
Green LED	anode	GP18	LINE OK, 220 to 330 ohm to GND
Fault-clear button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat

Device	Pin on device	Pico 2 GPIO	Notes
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use **3.3 V logic** on every GPIO. The only 5 V connection is the LCD backpack supply and the servo rail. The 1000 uF capacitor on the servo rail is required to stop the SG90 current spike from browning out the node.

Flash in BOOTSEL mode (hold BOOT, plug in USB) and copy the UF2 onto the **RP2350** mass-storage drive, or use **picotool**.

Memory Map Reference

Region	Address	Purpose
Bootrom	0x00000000	Immutable boot code
Flash/XIP	0x10000000	Vector table, code, rodata, data image
SRAM	0x20000000	Stack and writable state
CoreDebug DHCSR	0xE000EDF0	Anti-debug register read by the implant
Implant reserved sector	0x103FF000	Bot marker target (last flash sector)
Implant tick counter	0x20013714	Incremented once per <code>implant_tick</code>
Implant task count	0x20013710	Number of executed remote tasks
Implant check-in count	0x2001370C	Number of emitted check-in frames
Implant armed flag	0x20013CF7	Set when the payload handler arms
Implant checked-in flag	0x20013CF8	Set after a check-in frame is emitted
Implant check-in enable	0x20013CF9	Enables the check-in readiness test
Implant check-in gate	0x20013CFA	Gates the <code>C2V1</code> registration
Implant last task	0x20013CFB	Last executed task code
Implant marker gate	0x20013CFC	Gates the reserved-sector marker write
Implant task gate	0x20013CFD	Gates the remote task handler
Implant tasking enable	0x20013CFE	Enables the tasking readiness test
Control ready gate	0x20013CF3	Gates the sealed fault command path
Applied command	0x20013CF2	Command after a true verdict
Applied zone	0x20013CE2	Zone after a true verdict
Authorization ready gate	0x20013CEE	Gates the authorization check
Auth state record	0x200136CC	Anti-replay and state-tag record
Control field key	0x200131F4	Derived field key for the envelope

Region	Address	Purpose
Envelope workspace	0x200136E8	Sealed frame open workspace

The VA of any file offset is the file offset plus 0x10000000. Every defect is a file offset and a VA that differ by exactly that base.

Submission Format

Submit a folder containing:

- ACT-VII-Answers.md with all written answers;
- screenshots or terminal transcripts, including the anti-debug GDB session and the reserved-sector read;
- ACT-VII_fixed.bin and ACT-VII_fixed.uf2;
- the output of `python scripts/verify_ctf.py`;
- the original image SHA-256.

Success Criteria

You complete the challenge when you can prove all of the following:

- You can explain how the RP2350 reaches the controller code from reset.
- You can find and patch all four defect bytes and show the before/after values.
- You can explain the C2V1 check-in and why bot id 0xB7 marks the station.
- You can explain the benign task handler and why an obedience channel is a threat even when the tasks do no harm.
- You can explain the reserved-sector marker and why a firmware reflash does not remove the bot.
- You can explain the DHCSR anti-debug trap and show under GDB that you defeated it to observe the marker write.
- You can explain why unauthenticated and replayed fault and clear commands must be rejected, and why an authenticated wire does not protect an actuator from code on the same chip.
- You can export, convert, flash, and prove the corrected behavior on real hardware.
- `python scripts/verify_ctf.py` passes.

Academic Integrity

By submitting this CTF work, you certify that:

1. You used only the supplied training node, image, and lab interface.
2. You did not connect the challenge to a public network, an operational factory network, a manufacturing execution system, a building-management system, or any third-party device.
3. You understand that embedded reverse engineering and binary patching require explicit authorization in any real-world context.
4. You will report any discovered weakness responsibly to the course instructor.

The world is short on people who can read a stripped image and tell an honest byte from a lie. Treat that responsibility seriously: verify before you patch, patch before you trust, and never confuse a green lamp with a station that answers to someone else.

Reference Material

- ARM Cortex-M33 Technical Reference Manual
- ARMv8-M Architecture Reference Manual (CoreDebug DHCSR)
- RP2350 datasheet
- GDB documentation
- Ghidra documentation: <https://ghidra-sre.org/>
- Argon2 memory-hard function: <https://www.rfc-editor.org/rfc/rfc9106>
- ChaCha20-Poly1305 AEAD: <https://www.rfc-editor.org/rfc/rfc8439>
- PHC reference Argon2: <https://github.com/P-H-C/phc-winner-argon2>
- Project disassembly: `ACT-VII-main-disasm.txt`
- Machine verifier: `scripts/verify_ctf.py`